

OSGi Working Group  
OSGi Specification repo for  
`org.osgi.service.resourcemoni-`  
toring Release SNAPSHOT

September 2026



---

## Copyright © 2000, 2026 Eclipse Foundation

### LICENSE

#### Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

#### Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

#### Preface

#### Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

#### Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

---

---

# Table of Contents

|            |                                                   |          |
|------------|---------------------------------------------------|----------|
| <b>1</b>   | <b>Introduction</b>                               | <b>5</b> |
| 1.1        | Reader Level .....                                | 5        |
| 1.2        | Version Information .....                         | 5        |
| 1.3        | References .....                                  | 5        |
| <b>144</b> | <b>Resource Monitoring Specification</b>          | <b>7</b> |
| 144.1      | Introduction .....                                | 7        |
| 144.2      | Essentials .....                                  | 7        |
| 144.3      | Entities .....                                    | 7        |
| 144.4      | Operation Summary .....                           | 8        |
| 144.5      | Resource Context .....                            | 9        |
| 144.6      | System Resource Context .....                     | 9        |
| 144.7      | Framework Resource Context .....                  | 9        |
| 144.8      | Resource Monitor .....                            | 10       |
| 144.9      | Resource Monitor Factory .....                    | 10       |
| 144.10     | CPU Monitor .....                                 | 11       |
| 144.11     | Memory Monitor .....                              | 11       |
| 144.12     | Socket Monitor .....                              | 11       |
| 144.13     | Disk Storage Monitor .....                        | 12       |
| 144.14     | Thread Monitor .....                              | 12       |
| 144.15     | Resource Listener .....                           | 12       |
| 144.16     | Resource Event .....                              | 15       |
| 144.17     | Resource Context Listener .....                   | 16       |
| 144.18     | Resource Context Event .....                      | 16       |
| 144.19     | Resource Monitoring Service .....                 | 17       |
| 144.20     | Resource Monitoring Client .....                  | 17       |
| 144.21     | Security .....                                    | 17       |
| 144.22     | org.osgi.service.resourcemonitoring .....         | 17       |
| 144.23     | org.osgi.service.resourcemonitoring.monitor ..... | 29       |
| 144.24     | References .....                                  | 31       |

---

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

# 1 Introduction

This document contains a specification developed by the OSGi working group.

## 1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

## 1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

## 1.3 References

- [1] *OSGi Specifications*  
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

# 144 Resource Monitoring Specification

## Version 1.0

### 144.1 Introduction

Applications, executed on an OSGi platform, need hardware resources (CPU, memory, disk, storage space) and software resources (sockets, threads). As these resources are limited, applications have to share them in order to preserve system quality of service. This is a general fact in OSGi business cases where multiple bundles share the OSGi framework. This is especially the case when the framework is shared by distinct tenants, which are responsible for distinct set of bundles running with their own business logic and lifecycle.

The chapter defines an API for applications to monitor hardware resources consumed by any set of bundles. The bundle is the smallest unit that can be considered as a resource context, the entity that is monitored. Monitored data may enable applications to take decisions on management actions to apply. Resource management actions are mentioned as examples in this chapter, for example, actions on the lifecycle of components, bundles, the framework and the JVM, java threads, raise of exceptions.

### 144.2 Essentials

- *Monitoring* - Bundle execution resource usage is monitored.
- *Granular activation* - The resource monitoring service can be activated and deactivated per bundle or per bundle set.
- *Extensibility* - Five resource types are specified (CPU, memory, disk storage, alive thread and in-use sockets). The list of monitored resource types is extensible and query-able.
- *Eventing* - The resource monitoring service notifies interested entities of exceeded limits.

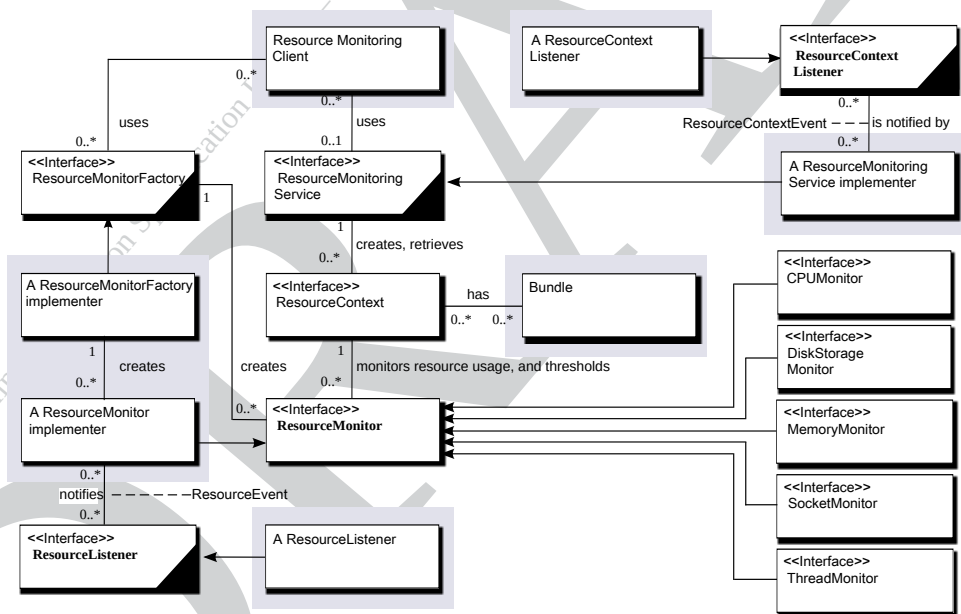
### 144.3 Entities

- *Resource Context* - A logical entity for resource accounting. A context may be related to a single bundle or a set of bundles.
- *System Resource Context* - Resource context of the core framework.
- *Platform Resource Context* - A Resource context monitoring the resource usage of the platform as a whole.
- *Resource Monitor* - Monitors the usage of a specific resource type for a specific Resource Context. Resource Monitors track resource usage. They hold Resource Thresholds instances. Resource Monitor object implementation may depend on standard or proprietary JVM APIs, and on operating system features.
- *Resource Monitor Factory* - A factory creating Resource Monitor instances for every Resource Context.
- *CPU Monitor* - Resource Monitor used to monitor CPU.
- *Memory Monitor* - Resource Monitor used to monitor memory.

- *Socket Monitor* - Resource Monitor used to monitor socket resource.
- *Disk Storage Monitor* - Resource Monitor for disk storage usage.
- *Thread Monitor* - Resource Monitor used to monitor alive Java Thread objects.
- *Resource Listener* - A Resource Listener receives resource threshold notifications.
- *Resource Event* - A Resource Event defines a notification to be synchronously sent to Resource Listener instances.
- *Resource Context Listener* - A Resource Context Listener receives notifications about resource context creation and configuration.
- *Resource Context Event* - A Resource Context Event defines a notification to be sent to Resource Context Listeners instances.
- *Resource Monitoring Service* - This is a singleton entity which manages Resource Context instances. It is used to create new Resource Context instances and to enumerate existing contexts.
- *Resource Monitoring Client* - Makes any decision to ensure the quality of the service of the system. They use the Resource Monitoring Service to create Resource Context instances. It configures them by adding bundles and Resource Monitors.

Figure 144.1

Resource monitoring class diagram specification.



## 144.4 Operation Summary

Resource Monitoring Clients use the Resource Monitoring Service service to create Resource Contexts. These clients set bundles or group of bundles to Resource Contexts. They also request every Resource Monitor Factory to create Resource Monitors for a resource type. These Resource Monitors are associated to a single Resource Context.

When activated, Resource Monitors provide the current resource usage per Resource Context. Then, they check whether the current resource usage is compatible with the thresholds held by their associated Resource Listeners. When one of these thresholds is violated, the related Resource Monitor notifies the Resource Listener holding this threshold.

The Resource Monitoring Service manages the set of Resource Contexts. Resource Contexts are persistent between platform restarts. Resource Context Listeners are notified when a Resource Context is created or deleted or when a Resource Context configuration (that is, adding or removing of bundle) is updated.

## 144.5 Resource Context

A [ResourceContext](#) instance is a logical entity used to account resource usage. Every Resource Context defines a bundle scope which can be either a single bundle or a set of bundles. Once the bundle scope is defined, resources used by those bundles are monitored through a set of per-resource-type Resource Monitor instances.

Resource Context instances are persistent. The persistence of those instances is directly managed by the Resource Monitoring Service instance.

Each Resource Context is uniquely identified by a name. It can be retrieved through the [getName\(\)](#) method. It can not be changed, that is it is definitively set when the Resource Context instance is created.

The Resource Context bundle scope is retrieved through the [getBundleIds\(\)](#) method. This bundle scope can be extended through the [addBundle\(long\)](#) method. Bundles can also be removed from a Resource Context through the [removeBundle\(long,ResourceContext\)](#) method. For this last method, a Resource Context instance MAY be specified in order to associate the removed bundle to another Resource Context instance.

Resource Monitor instances are retrieved through [getMonitor\(String\)](#) method or the [getMonitors\(\)](#) method. The list of available resource types is retrieved through the Resource Monitoring Service singleton instance.

Resource Monitor instances are added to and removed from a Resource Context instance by calling either [addResourceMonitor\(ResourceMonitor\)](#) method or [removeResourceMonitor\(ResourceMonitor\)](#) method. Both methods SHOULD only be called by [ResourceMonitorFactory](#) instances (see [createResourceMonitor\(ResourceContext\)](#) method).

A Resource Context is retrieved through the Resource Monitoring Service service.

A Resource Context instance can be deleted through [removeContext\(ResourceContext\)](#) method. The Resource Context input argument then defines a destination Resource Context instance for the bundles belonging to the to-be-removed Resource Context instance.

## 144.6 System Resource Context

The System Resource Context is the Resource Context of the execution environment for the running OSGi bundles. It includes the resources of bundle "0". It is retrieved through the Resource Monitoring Service service.

The name of this context is "system". See [SYSTEM\\_CONTEXT\\_NAME](#).

## 144.7 Framework Resource Context

The Framework Resource Context is a Resource Context monitoring resources of the platform as a whole. It is retrieved through the Resource Monitoring Service service. This Resource Context holds all hosted bundles allowing access to the whole platform resource consumption.

The name of this context is "framework". See [FRAMEWORK\\_CONTEXT\\_NAME](#).

# 144.8Resource Monitor

A [ResourceMonitor](#) instance monitors a resource type consumed by the bundles of a specific Resource Context instance.

A Resource Context instance holds at most one Resource Monitor instance per monitor-able resource type. Resource Monitor instances are retrieved through their related Resource Context instance. Resource Monitor instances give access to their related Resource Context instance through a call to See [getContext\(\)](#) method.

The monitored resource type is retrieved through the [getResourceType\(\)](#) method.

The current usage of a resource consumed by a Resource Context instance is given through the [getUsage\(\)](#) method. This method returns a Java Object to be casted to the appropriate Java object type depending on the Resource type. The next table provides the expected Java Object type for each specified resource type:

Table 144.1Table of resource types.

| Type of Resource   | Expected java Object type | Value description                            |
|--------------------|---------------------------|----------------------------------------------|
| CPU                | Long                      | Cumulative CPU time in ns.                   |
| Memory             | Long                      | Allocated memory in bytes.                   |
| Threads            | Long                      | Number of alive thread.                      |
| Socket             | Long                      | Number of in-use socket.                     |
| Disk storage space | Long                      | Bytes on the bundle persistent storage area. |

For example, for a [MemoryMonitor](#) instance, a call to [getUsage\(\)](#) returns a Long java object indicating the amount of memory the related Resource Context instance is consuming.

A Resource Monitor instance is enabled and disabled through [enable\(\)](#) and [disable\(\)](#) methods. The state (enabled or disabled) of a Resource Monitor is retrieved through a call to [isEnabled\(\)](#) method. Enable and disable monitoring mechanisms on-the-fly on localized set of bundles may be crucial for performance issues. See [1] *Adaptive Monitoring of End-user OSGi based Home Boxes* .

A Resource Monitor instance can also be deleted ([delete\(\)](#) method). [isDeleted\(\)](#) method returns true if the ResourceMonitor instance has been deleted.

Five types of Resource Monitor are specified:

- CPU Monitor
- Memory Monitor
- Socket Monitor
- Disk Storage Monitor
- Thread Monitor

The support of any Resource Monitor is optional. This list MAY be extended by the solution vendor. The list of the types that are supported on the OSGi platform can be computed by querying [ResourceMonitorFactory](#) services. Resource monitoring algorithms may vary with factories, see [2] *Memory Monitoring in a Multi-tenant OSGi Execution Environment* . They are out of the scope of this specification.

# 144.9Resource Monitor Factory

A [ResourceMonitorFactory](#) is a service that provides Resource Monitor instances of a specific resource type (for example, [CPUMonitor](#) , [MemoryMonitor](#) , etc.) for every Resource Context.

Every Resource Monitor Factory service is registered with the `org.osgi.resourcemonitoring.ResourceType` mandatory property, see [RESOURCE\\_TYPE\\_PROPERTY](#). This property indicates which type of Resource Monitor a Resource Monitor Factory is able to create. The type can also be retrieved through a call to `getType()`. The type MUST be unique (two Resource Monitor Factory services MUST not have the same type).

New Resource Monitor instances are created by a call to `createResourceMonitor(ResourceContext)`. This method returns a new Resource Monitor instance associated to the provided Resource Context instance. The ResourceMonitorFactory MUST call `addResourceMonitor(ResourceMonitor)` to associate the newly created ResourceMonitor with the provided ResourceContext instance. The newly created Resource Monitor is disabled, that is, it is initially not monitoring the Resource Context resource consumption. It can be activated through a call to `enable()`.

Resource Monitor instances are deleted by calling `delete()` method.

A Resource Monitor instance MUST only be created through its [ResourceMonitorFactory](#).

Resource Monitor Factory instances should be only used by the Resource Monitoring Service singleton instance. The Resource Monitoring Service singleton instance performs a service lookup on all existing Resource Monitor Factories. It uses a Resource Monitor Factory instance when it has to create a new Resource Context instance and their associated Resource Monitor instances.

## 144.10 CPU Monitor

A [CPUMonitor](#) instance is a Resource Monitor used to monitor the CPU usage of the bundles belonging to a Resource Context.

CPU usage and thresholds are expressed as a cumulative number of nanoseconds (long). The encapsulated value can be retrieved with the `getCPUUsage()` method.

In case where a threshold is reached, the CPU Monitor instance generates an event triggering Resource Monitoring Clients defined corrective actions (for example, decrease thread priority).

## 144.11 Memory Monitor

A [MemoryMonitor](#) instance monitors and limits the memory used by the bundles of a Resource Context instance.

Memory is accounted as bytes. Memory usage and thresholds are long java objects. The encapsulated value can be retrieved through the `getMemoryUsage()` method.

When an error threshold is reached, the next memory allocation MAY be prevented by the system and MAY throw a specific Exception in the associated context.

## 144.12 Socket Monitor

A [SocketMonitor](#) instance monitors and limits the number of existing sockets (for example, TCP, UDP) which are considered to be in use (for example, listening for incoming packet, bound, or sending outgoing packets).

A Socket is considered to be in-use state when a native socket file descriptor is created. It leaves this state when this socket file descriptor is deleted.

The number of in-use sockets is a long. The encapsulated value can be retrieved using `getSocketUsage()` method.

When an [ERROR](#) threshold is reached, the next socket file descriptor creation in the associated context MAY throw a `SocketException`.

## 144.13 Disk Storage Monitor

A [DiskStorageMonitor](#) instance monitors and limits the use of persistent storage within Bundle Persistent Storage Area a Resource Context (the bundles actually belonging to it) consumes.

Disk Storage is expressed as a number of bytes of type `long`. The encapsulated value can be retrieved using [getUsedDiskStorage\(\)](#) method.

An `IOException` MAY be thrown in the associated context when an [ERROR](#) threshold is reached.

## 144.14 Thread Monitor

A [ThreadMonitor](#) instance monitors and limits the number of alive Java Thread objects for a Resource Context instance. A Thread is considered to be alive when it is in the `RUNNABLE`, `BLOCKED`, `WAITING` or `TIMED_WAITING` thread state.

Usage and thresholds are Java `int` objects. The encapsulated value can be retrieved using [getAliveThreads\(\)](#) method.

When an [ERROR](#) threshold is reached, any further thread activation will be prevented in the associated context. An `InternalError` exception MAY also be thrown in the associated context.

## 144.15 Resource Listener

A [ResourceListener](#) receives notifications about resource usage for a specific Resource Context and a specific type of resource. A notification will be sent to a Resource Listener when one of its thresholds is violated.

A Resource Listener holds two types of threshold:

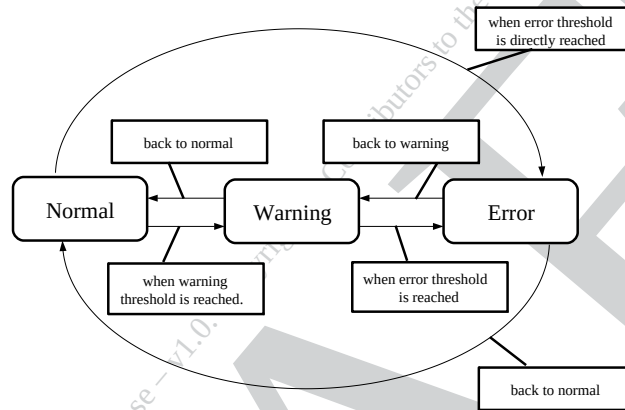
- A lower threshold type. This kind of threshold is reached when the monitored resource usage decreases below the threshold.
- An upper threshold type. An upper threshold is reached when the monitored resource usage exceeds this threshold.

Each of them have two levels:

- a [WARNING](#) level.
- an [ERROR](#) level.

A threshold has the following state diagram, which transitions are associated to events:

Figure 144.2 Threshold state diagram.



A threshold state depends on the current consumption of resource and the type of threshold (upper or lower threshold).

A Resource Listener is registered as an OSGi service. The implementer must provide the two following mandatory properties:

- **RESOURCE\_CONTEXT** property – a String defining the name of Resource Context for which the Listener want to receive threshold notifications.
- **RESOURCE\_TYPE** property – a String defining which type of resource the listener wants to monitor.

It also has to provide at least one of these four properties when registered as an OSGi service:

- **UPPER\_WARNING\_THRESHOLD**
- **UPPER\_ERROR\_THRESHOLD**
- **LOWER\_WARNING\_THRESHOLD**
- **LOWER\_ERROR\_THRESHOLD**

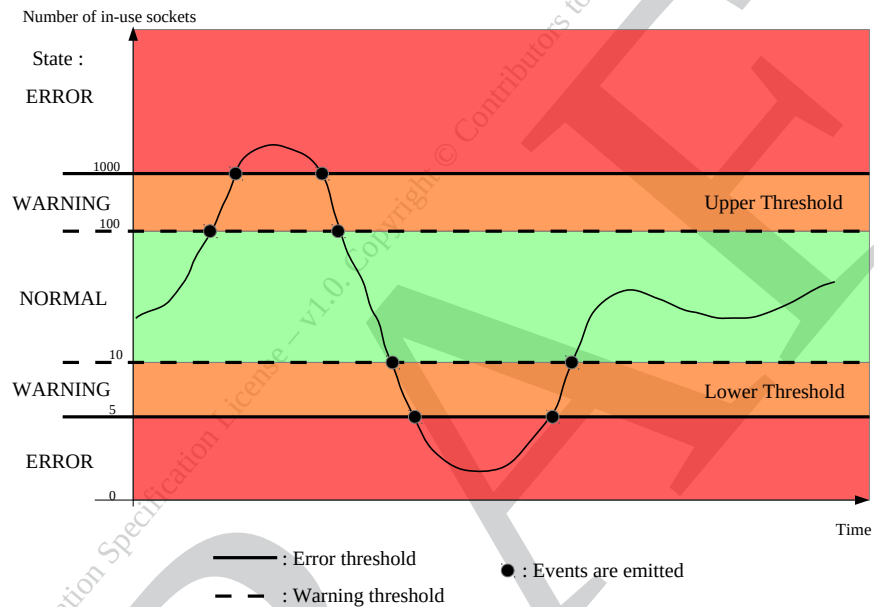
These properties are mapped to the four types of threshold values a Resource Listener may support. The service properties are used to notify the associated Resource Monitor when one of these threshold values is modified.

Threshold values can also be retrieved through a set of getter methods. All of these methods returns a Comparable object used by the associated Resource Monitor in order to determine the current state of the current usage.

**RESOURCE\_CONTEXT** and **RESOURCE\_TYPE** properties are used by Resource Monitors to identify their associated Resource Listeners. Once associated, a Resource Monitor retrieves the threshold settings using service properties. When one of its thresholds is reached, the Resource Monitor calls `notify(ResourceEvent)`.

Two examples of resource consumption are explained below, first with in-use sockets monitoring, second with CPU monitoring. The next picture shows the state diagram of the number of in-use state socket over the time.

Figure 144.3 Number of in-use sockets over the time.



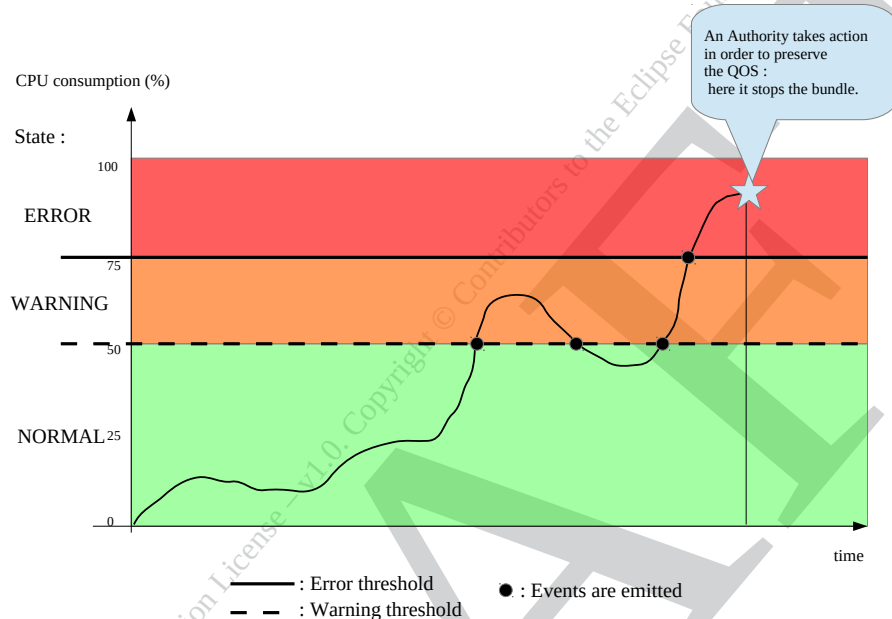
In our example, the lower warning threshold and the lower error threshold of the Resource Listener are respectively set to 10 and 5. When the number of in-use sockets decreases under 10, the usage goes from the **NORMAL** state to the **WARNING** state and the Resource Listener receives a **WARNING** event. If the number of in-use state sockets decreases again and goes down to 5, the usage goes from the **WARNING** state to the **ERROR** state and the Resource Listener receives a **ERROR** Resource Event.

The upper threshold is also set. The upper warning threshold and the upper error threshold are respectively set to 100 and 1000 in-use state sockets. When the number of sockets reaches 100, the usage goes from the **NORMAL** state to the **WARNING** state and the Resource Listener receives a **WARNING** Resource Event. If this number is still increasing and exceeds 1000, then the usage goes from the **WARNING** state to the **ERROR** state and the Resource Listener receives an **ERROR** Resource Event.

This is a typical use case for a Java Web server. Indeed, one of the most important quality of service indicator is the number of in-use state sockets a java web server is handling. A low number of in-use state sockets may indicate the java web server encounters network problems. On the contrary, a high number of in-use state socket may be the result of an external network attack or it could also indicates the java web server is overused and its administrator should take actions to load-balance the charge to another java web server instance.

For other resource types, only upper thresholds may be useful. The next diagram shows the CPU consumption a Resource Context is using over the time:

Figure 144.4 CPU consumption (%) over the time – Upper Threshold.



In this example, only the upper threshold is set. The upper warning threshold is set to 50%, the error one is set to 75%. CPU consumption fluctuates between 0 and 50%, the usage is in the **NORMAL** state. Then it increases and reaches 50%. The usage then goes from the **NORMAL** state to the **WARNING** state and the Resource Listener holding the threshold receives a **WARNING** Resource Event.

Afterwards, CPU consumption decreases under 50%; the usage goes from the **WARNING** state to the **NORMAL** state. The related Resource listener receives a **NORMAL** Resource Event.

It then increases again and exceeds 50%. The usage goes to the **WARNING** state. CPU consumption is still increasing and exceeds 75%. At this moment, the usage goes from the **WARNING** state to the **ERROR** state and the related Resource Listener receives an **ERROR** Resource Event.

After some seconds in the **ERROR** state, the Resource Listener implementation stops the bundle in order to preserve the quality of service.

The choice of the type of threshold (lower or upper, or both of them) depends on the type of resource and the needs of the Resource Monitoring Clients providing the Resource Listener. Other resources like the free memory may take advantage of a lower threshold.

## 144.16 Resource Event

A **ResourceEvent** instance is an event synchronously sent to a Resource Listener when one of its thresholds is reached. This event is notified to a Resource Listener through a call to `ResourceListener.notify(ResourceEvent)`.

A Resource Event has a type among the following ones:

- **ERROR** – The resource consumption reaches either the upper or the lower error threshold of the Resource Listener receiving this event.

- **WARNING** – The resource consumption reaches either the upper or the lower warning threshold of the Resource Listener receiving this event.
- **NORMAL** – The resource consumption is back from warning or error state to normal state.

The Resource Listener instance analyzes this event by calling the following methods:

- `getValue()` method returns the resource consumption at the time when the Resource Event instance was generated.
- `isUpperThreshold()` method returns true if the reached threshold is an upper threshold type. If this method returns false, this is a lower threshold.
- `getType()` method indicates the state (WARNING, ERROR, or NORMAL) of the resource usage.
- `getContext()` method returns the Resource Context instance related to this event. The Resource Listener can use it to retrieve the Resource Monitor instance. For example, `event.getContext().getMonitor(event.getResourceType())`.

## 144.17 Resource Context Listener

A `ResourceContextListener` instance receives notifications about Resource Context lifecycle and configuration.

A notification will be sent when:

- A Resource Context is created.
- A Resource Context is updated, that is, a bundle has been added or removed from a Resource Context instance.
- A Resource Context is deleted.

An application which is interested in notifications has to register a Resource Context Listener instance as an OSGi service. The application may provide a set of properties at registration time to reduce the number of notifications a Resource Listener instance will receive. The available property is:

- **RESOURCE\_CONTEXT** property – An array of String defining the name of Resource Context instances. If defined, a Resource Listener instance will only receive notifications related to these specified Resource Context instances.

A Resource Context Listener instance is notified through a call to `notify(ResourceContextEvent)` method.

## 144.18 Resource Context Event

A `ResourceContextEvent` instance is an event sent to Resource Context Listener instances through a call to the `notify(ResourceContextEvent)` method.

A Resource Context Event has a type among the four following ones:

- **RESOURCE\_CONTEXT\_CREATED** – A new Resource Context instance has been created.
- **RESOURCE\_CONTEXT\_REMOVED** – A Resource Context instance has been deleted.
- **BUNDLE\_ADDED** – A bundle has been added in the scope of a Resource Context instance.
- **BUNDLE\_REMOVED** – A bundle has been removed from the scope of a Resource Context instance.

In the case of a **RESOURCE\_CONTEXT\_CREATED** event or a **RESOURCE\_CONTEXT\_REMOVED** event, a call to `getContext()` returns the targeted Resource Context instance.

In the case of a `BUNDLE_ADDED` type or `BUNDLE_REMOVED` type, `getBundleId()` returns the id of the bundle to be added to or removed from. The related Resource Context instance is given by a call to `getContext()`.

## 144.19 Resource Monitoring Service

The `ResourceMonitoringService` manages the Resource Context instances. The Resource Monitoring Service is available through the OSGi service registry.

This service holds the existing Resource Context instances. Resource Context instances are created by calling the `createContext(String,ResourceContext)` method. The caller provides a context name as a string and optionally a template as a ResourceContext object.

The list of existing Resource Context instances can be retrieved through the following methods:

- `getContext(String)` – returns the ResourceContext with the specified resource context name.
- `getContext(long)` – returns the ResourceContext associated to the provided bundle id.
- `listContext()` – retrieve all existing Resource Context instances as an array.

The Resource Monitoring Service singleton manages the persistence of the Resource Context instances. The following properties are stored:

- name of the Resource Context.
- list of the bundles belonging to the Resource Context.
- list of the Resource Monitor instances. For each one: the sampling period, and the monitoring period.

The way the Resource Monitoring Service persists the Resource Context instances is implementation specific. The implementer is free to use any file format and file location it wants. At startup, the Resource Monitoring Service will load the persisted Resource Context instances to restore the state prior to shutdown.

## 144.20 Resource Monitoring Client

A Resource Monitoring Client uses the Resource Monitoring Service singleton instance to apply Resource Monitoring policies. These entities MAY:

- create and configure Resource Context instances (resource thresholds, bundle scope)
- take any decisions (stop a bundle, uninstall a bundle) if a Resource Context exceeds resource limit.

These policies are out of the scope of this specification.

## 144.21 Security

It is recommended that `ServicePermission[ResourceMonitoringService|ResourceMonitoringFactory|ResourceListener, REGISTER|GET]` be used sparingly and only for bundles that are trusted.

## 144.22 org.osgi.service.resourcemonitoring

Resource Monitoring Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.resourcemonitoring; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.resourcemonitoring; version="[1.0,1.1)"}
```

## 144.22.1

### Summary

- **ResourceContext** - Logical entity for resource accounting.
- **ResourceContextEvent** - A Resource Context Event instance is an event sent to Resource Context Listener instances through a call to {@link ResourceContextListener#notify(ResourceContextEvent)} method.
- **ResourceContextException** - Resource Context Exception.
- **ResourceContextListener** - A ResourceContextListener is notified whenever:
  - a {@link ResourceContext} is created or deleted.
  - a bundle is added or removed from a {@link ResourceContext}.
- **ResourceEvent** - An event is sent to a {@link ResourceListener} when resource usage violates one of their thresholds.
- **ResourceListener** - A ResourceListener is an OSGi service which is notified when a Resource Context violates one of the threshold defined by the listener.
- **ResourceMonitor** - Representation of the state of a resource for a resource context.
- **ResourceMonitorException** - Resource Monitor Exception reports an invalid usage of a monitor.
- **ResourceMonitorFactory** - A Resource Monitor Factory is a service that provides Resource Monitor instances of a specific resource type (for example, CPUMonitor, MemoryMonitor...) for every Resource Context.
- **ResourceMonitoringService** - It manages the Resource Context instances.

## 144.22.2

### public abstract ResourceContext

Logical entity for resource accounting. A resource context has a group of member bundles, and a bundle can be a member of 0 or 1 resource context.

Resource Monitoring Clients can use the {@link ResourceMonitoringService#createContext(String, ResourceContext)} method to create ResourceContext instances.

Resource Monitoring Clients can use the {@link #getMonitor(String)} method to get {@link ResourceMonitor} instances for the supported resource types. These instances can then be used to monitor the usage of the resources, or the set usage limits.

ResourceContexts are retrieved through the {@link ResourceMonitoringService} OSGi service.

### 144.22.2.1

#### public abstract void addBundle(long bundleId) throws ResourceContextException

*bundleId* The bundle to add to this resource context

- Adds a bundle to the resource context. The bundle will be a member of the context until it is uninstalled, or explicitly removed from the context with {@link #removeBundle(long)} method or {@link #removeBundle(long, ResourceContext)} method.

Resources previously allocated by this bundle (in another resource context) will not be moved to this resource context. The change applies only for future allocations.

A {@link ResourceContextEvent} with type {@link ResourceContextEvent#BUNDLE\_ADDED} will be sent.

*Throws* ResourceContextException– For example, when the bundle can't be added to the ResourceContext.

**144.22.2.2      public abstract void addResourceMonitor(ResourceMonitor<?> resourceMonitor) throws ResourceContextException**

*resourceMonitor* resourceMonitor instance to be added

- Adds a new ResourceMonitor instance monitoring resource for this resource context. This method should be called only by ResourceMonitorFactory instance.

*Throws* ResourceContextException– For example, when the monitor can't be added to the ResourceContext.

**144.22.2.3      public abstract boolean equals(Object resourceContext)**

*resourceContext* resource context

- A ResourceContext rc1 is equals to ResourceContext rc2 if rc1.getName() is equals to rc2.getName().

*Returns* true if getName().equals(resourceContext.getName())

**144.22.2.4      public abstract long[][] getBundleIds()**

- Returns the bundle identifiers belonging to this Resource Context.

*Returns* An array of {@link Bundle} objects, or an empty array if no bundles are currently members of this context

**144.22.2.5      public abstract ResourceMonitor<?> getMonitor(String resourceType) throws ResourceContextException**

*resourceType* The resource type, for which a resource monitor is requested

- Returns a ResourceMonitor instance for the specified resource type. If the {@link ResourceMonitoringService} implementation does not support this resource type, null is returned

*Returns* A ResourceMonitor instance, or null, if this resource type is not supported

*Throws* ResourceContextException– For example, when the monitor(s) can't be retrieved from the ResourceContext.

**144.22.2.6      public abstract ResourceMonitor<?>[][] getMonitors() throws ResourceContextException**

- Retrieves all the existing ResourceMonitor belonging to this context.

*Returns* an array of ResourceMonitor. May be empty if no ResourceMonitor

*Throws* ResourceContextException– For example, when the monitor(s) can't be retrieved from the ResourceContext.

**144.22.2.7      public abstract String getName()**

- Returns the name of the resource context. Resource context names are unique within a framework instance.

*Returns* The resource context name

**144.22.2.8      public abstract int hashCode()**

- Retrieves the hashCode value of a ResourceContext. The hashCode value of a ResourceContext is only based on the hashCode value of the name of the context.

*Returns* hashCode

**144.22.2.9      public abstract void removeBundle(long bundleId) throws ResourceContextException**

*bundleId* bundle identifier

- Removes the bundle identified by `bundleId` from the Resource Context. The bundle is no longer to this Resource Context.

*Throws* `ResourceContextException`– For example, when the bundle can't be removed from the Resource-Context.

#### 144.22.2.10 **public abstract void removeBundle(long bundleId, ResourceContext destination) throws ResourceContextException**

*bundleId* the identifier of the bundle to be removed from the Resource Context

*destination* A resource context in which to add the bundle, after removing it from this context. If no destination is provided (that is null), the bundle is not associated to a new Resource Context.

- Removes the bundle from this resource context. If a destination context is specified, the bundle will be added in it.

Resources previously allocated by this bundle will not be removed from the resource context. The change applies only for future allocations.

A {@link ResourceContextEvent} with type {@link ResourceContextEvent#BUNDLE\_REMOVED} will be sent.

*Throws* `ResourceContextException`– For example, when the bundle can't be removed from the Resource-Context.

#### 144.22.2.11 **public abstract void removeContext(ResourceContext destination) throws ResourceContextException**

*destination* The {@link ResourceContext} where the resources currently allocated by this resource context will be moved.

- Removes a resource context. All resources allocated in this resource context will be moved to the destination context. If destination is null, these resources will no longer be monitored.

A {@link ResourceContextEvent} with type {@link ResourceContextEvent#RESOURCE\_CONTEXT\_REMOVED} will be sent.

*Throws* `ResourceContextException`– For example, when the resource context can't be removed.

#### 144.22.2.12 **public abstract void removeResourceMonitor(ResourceMonitor<?> resourceMonitor) throws ResourceContextException**

*resourceMonitor* resource monitor instance to be removed

- Removes a ResourceMonitor instance from the context.

*Throws* `ResourceContextException`– For example, when the monitor can't be removed from the Resource-Context.

### 144.22.3 **public class ResourceContextEvent**

A Resource Context Event instance is an event sent to Resource Context Listener instances through a call to {@link ResourceContextListener#notify(ResourceContextEvent)} method. A Resource Context Event has a type among the four following ones:

- {@link #RESOURCE\_CONTEXT\_CREATED} – A new Resource Context instance has been created.
- {@link #RESOURCE\_CONTEXT\_REMOVED} – A Resource Context instance has been deleted.
- {@link #BUNDLE\_ADDED} – A bundle has been added in the scope of a Resource Context instance.
- {@link #BUNDLE\_REMOVED} – A bundle has been removed from the scope of a Resource Context instance.

- 144.22.3.1**      **public static final int BUNDLE\_ADDED = 2**  
 A bundle has been added to e {@link ResourceContext}  
 The {@link ResourceContext#addBundle(long)} method has been invoked
- 144.22.3.2**      **public static final int BUNDLE\_REMOVED = 3**  
 A bundle has been removed from a {@link ResourceContext}  
 {@link ResourceContext#removeBundle(long)} method or {@link ResourceContext#removeBundle(long, ResourceContext)} method have been invoked, or the bundle has been uninstalled
- 144.22.3.3**      **public static final int RESOURCE\_CONTEXT\_CREATED = 0**  
 A new {@link ResourceContext} has been created.  
 The {@link ResourceMonitoringService#createContext(String, ResourceContext)} method has been invoked.
- 144.22.3.4**      **public static final int RESOURCE\_CONTEXT\_REMOVED = 1**  
 A {@link ResourceContext} has been removed  
 The {@link ResourceContext#removeContext(ResourceContext)} method has been invoked
- 144.22.3.5**      **public ResourceContextEvent(int pType, ResourceContext pResourceContext)**  
*pType* event type  
*pResourceContext* context  
 □ Create a new ResourceContextEvent. This constructor should be used when the type of the event is either {@link #RESOURCE\_CONTEXT\_CREATED} or {@link #RESOURCE\_CONTEXT\_REMOVED}.
- 144.22.3.6**      **public ResourceContextEvent(int pType, ResourceContext pResourceContext, long pBundleId)**  
*pType* event type  
*pResourceContext* context  
*pBundleId* bundle  
 □ Create a new ResourceContextEvent. This constructor should be used when the type of the event is either {@link #BUNDLE\_ADDED} or {@link #BUNDLE\_REMOVED}.
- 144.22.3.7**      **public boolean equals(Object var0)**
- 144.22.3.8**      **public long getBundleId()**  
 Retrieves the identifier of the bundle being added to or removed from the Resource Context.  
 This method returns a valid value only when {@link #getType()} returns:
- {@link #BUNDLE\_ADDED}
  - {@link #BUNDLE\_REMOVED}
- Returns* the bundle id or -1 (invalid value).
- 144.22.3.9**      **public ResourceContext getContext()**  
 □ Retrieves the Resource Context associated to this event  
*Returns* Resource Context.
- 144.22.3.10**      **public int getType()**  
 □ Retrieves the type of this Resource Context Event.

*Returns* the type of the event. One of:

- {@link #RESOURCE\_CONTEXT\_CREATED}
- {@link #RESOURCE\_CONTEXT\_REMOVED}
- {@link #BUNDLE\_ADDED}
- {@link #BUNDLE\_REMOVED}

**144.22.3.11**      **public int hashCode()**

**144.22.3.12**      **public String toString()**

## **144.22.4**      **public class ResourceContextException** **extends Exception**

Resource Context Exception.

**144.22.4.1**      **public ResourceContextException(String msg)**

*msg* message

- Create a new ResourceContextException

**144.22.4.2**      **public ResourceContextException(String msg, Throwable t)**

*msg* message

*t* exception

- Create a new ResourceContextException

## **144.22.5**      **public abstract ResourceContextListener**

A ResourceContextListener is notified whenever:

- a {@link ResourceContext} is created or deleted.
- a bundle is added or removed from a {@link ResourceContext}.

A ResourceContextListener is registered as an OSGi service. At registration time, the following property may be provided:

- the {@link #RESOURCE\_CONTEXT} property which limits the Resource Context for which notifications will be received. This property can be either a String value or an array of String. If this property is not set, the Resource Context Listener receives events from all the Resource Context.

**144.22.5.1**      **public static final String RESOURCE\_CONTEXT = "resource.context"**

Property specifying the {@link ResourceContext}(s) for which a notification will be received by this listener.

The property value is either a string (i.e the name of the {@link ResourceContext}) and an array of string (several {@link ResourceContext}).

**144.22.5.2**      **public abstract void notify(ResourceContextEvent event)**

*event* event.

- Notify this listener about a {@link ResourceContext} events.

## **144.22.6**      **public class ResourceEvent<T>**

<T> The type for the Resource.

An event is sent to a {@link ResourceListener} when resource usage violates one of their thresholds. ResourceEvent objects are delivered synchronously to all matching {@link ResourceListener} services. A typed code is used to identify the event.

*See Also* ResourceListener

#### 144.22.6.1 **public static final int ERROR = 2**

Type of ResourceEvent indicating a threshold goes to the ERROR state.

#### 144.22.6.2 **public static final int NORMAL = 0**

Type of ResourceEvent indicating a threshold goes to the NORMAL state.

#### 144.22.6.3 **public static final int WARNING = 1**

Type of ResourceEvent indicating a threshold goes to the WARNING state.

#### 144.22.6.4 **public ResourceEvent(int pType, ResourceContext pContext, boolean pIsUpperThreshold, Comparable<T> pValue)**

*pType* the event type

*pContext* the resource context

*pIsUpperThreshold* whether it is an upper threshold

*pValue* the value

- Creates a new ResourceEvent.

#### 144.22.6.5 **public boolean equals(Object var0)**

#### 144.22.6.6 **public ResourceContext getContext()**

- Returns the resource context that caused the event.

*Returns* The resource context that caused the event.

#### 144.22.6.7 **public int getType()**

- Returns the event type. The type values are:
  - {@link #NORMAL}
  - {@link #WARNING}
  - {@link #ERROR}

*Returns* The event type

#### 144.22.6.8 **public Comparable<T> getValue()**

- Returns the resource consumption value. Relevant only for event types {@link #NORMAL}, {@link #WARNING} and {@link #ERROR}.

*Returns* the resource consumption value, or null if a resource monitor is not relevant.

#### 144.22.6.9 **public int hashCode()**

#### 144.22.6.10 **public boolean isUpperThreshold()**

- Returns true if the threshold triggering this event is an upper threshold. This method is only used when {@link #getType()} returns {@link #NORMAL}, {@link #WARNING} or {@link #ERROR}.

*Returns* true if it is an upper threshold.

**144.22.6.11**      **public String toString()**

## **144.22.7**      **public abstract ResourceListener<T>**

<T> The type for the Resource.

A ResourceListener is an OSGi service which is notified when a Resource Context violates one of the threshold defined by the listener.

Every ResourceListener is associated to a specific Resource Context and a specific Resource type. It defines two types of thresholds: a lower and a upper. A lower threshold is reached when the resource usage decreases below the threshold. On the contrary, an upper threshold is reached when the resource usage exceeds the threshold.

Both lower or upper threshold are two levels: a warning level and error level. The warning level indicates the resource usage becomes to be critical but are still acceptable. The error level indicates the resource usage is now critical for the overall system and actions should be taken.

A Resource Listener is registered with these two mandatory properties:

- {@link #RESOURCE\_CONTEXT} which defines the ResourceContext associated to this Listener
- {@link #RESOURCE\_TYPE} which the type of resource

The next optional properties are used to specify threshold values. A ResourceListener must at least provides one of them:

- {@link ResourceListener#UPPER\_WARNING\_THRESHOLD}
- {@link ResourceListener#UPPER\_ERROR\_THRESHOLD}
- {@link ResourceListener#LOWER\_WARNING\_THRESHOLD}
- {@link ResourceListener#LOWER\_ERROR\_THRESHOLD}

These threshold values can also be retrieved through methods.

Resource Listeners are associated to a Resource Context and a Resource Monitor based on the {@link #RESOURCE\_CONTEXT} property and the {@link #RESOURCE\_TYPE} property (both of them are mandatory at registration time).

Once associated, the ResourceMonitor gets the threshold values through the service properties (i.e {@link #UPPER\_WARNING\_THRESHOLD}, {@link #UPPER\_ERROR\_THRESHOLD}, {@link #LOWER\_WARNING\_THRESHOLD} and {@link #LOWER\_WARNING\_THRESHOLD}) and store them. Once it detects a new resource consumption, it compares the new resource usage value with the thresholds provided by the Resource Listener. If the resource usage violates one of these thresholds, the Resource Monitor notifies the {@link ResourceListener} through a call to {@link #notify(ResourceEvent)}.

A ResourceMonitor tracks threshold value modification by using a {@link ServiceListener}.

**144.22.7.1**      **public static final String LOWER\_ERROR\_THRESHOLD = "lower.error.threshold"**

Optional property defining the value of the lower error threshold.

**144.22.7.2**      **public static final String LOWER\_WARNING\_THRESHOLD = "lower.warning.threshold"**

Optional property defining the value of the lower warning threshold.

**144.22.7.3**      **public static final String RESOURCE\_CONTEXT = "resource.context"**

Mandatory property specifying the Resource Context associated with the listener.

**144.22.7.4**      **public static final String RESOURCE\_TYPE = "resource.type"**

Mandatory property defining the type of Resource (i.e the ResourceMonitor) associated to this Listener.

**144.22.7.5**      **public static final String UPPER\_ERROR\_THRESHOLD = "upper.error.threshold"**

Optional property defining the value of the upper error threshold.

**144.22.7.6**      **public static final String UPPER\_WARNING\_THRESHOLD = "upper.warning.threshold"**

Optional property defining the value of the upper warning threshold.

**144.22.7.7**      **public abstract Comparable<T> getLowerErrorThreshold()**

- Retrieves the lower error threshold value set by the listener. If the resource usage decreases under this threshold, the {@link #notify(ResourceEvent)} will be called. The provided {@link ResourceEvent} then indicates the ERROR state is reached.

*Returns* a comparable object or null if no threshold is set.

**144.22.7.8**      **public abstract Comparable<T> getLowerWarningThreshold()**

- Retrieves the lower warning threshold value set by the listener. If the resource usage decreases under this threshold value, the {@link #notify(ResourceEvent)} will be called. The provided ResourceEvent then indicates the WARNING state is reached.

*Returns* a comparable object or null if no threshold is set.

**144.22.7.9**      **public abstract Comparable<T> getUpperErrorThreshold()**

- Retrieves the upper error threshold value set by this listener. If the resource usage exceeds this threshold, the {@link #notify(ResourceEvent)} will be called. The provided {@link ResourceEvent} then indicates the ERROR state is reached.

*Returns* a comparable object or null if no threshold is reached.

**144.22.7.10**      **public abstract Comparable<T> getUpperWarningThreshold()**

- Retrieves the upper warning threshold value set by this listener. If the resource usage exceeds this threshold, the {@link #notify(ResourceEvent)} method will be called. The provided {@link ResourceEvent} then indicates the WARNING state is reached.

*Returns* a comparable object or null if no threshold is reached.

**144.22.7.11**      **public abstract void notify(ResourceEvent<T> event)**

*event* The {@link ResourceEvent} object

- Receives a resource monitoring notification

**144.22.8**      **public abstract ResourceMonitor<T>**

*<T>* The type for the Resource.

Representation of the state of a resource for a resource context.

ResourceMonitor objects are returned by the {@link ResourceContext#getMonitor(String)} method.

The ResourceMonitor object may be used to:

- Enable/Disable the monitoring of the corresponding resource type for the corresponding resource context
- View the current usage of the resource by this resource context

A resource monitor can have a sampling period, a monitored period, or both. For example, for CPU monitoring, the resource monitor implementation can get the CPU usage of the running threads once per minute, and calculate the CPU usage per context in percentages based on the last ten such measurements. This could make a 60 000 milliseconds sampling period, and a 600 000 milliseconds monitored period.

**144.22.8.1      public abstract void delete() throws ResourceMonitorException**

- Disable and delete this instance of Resource Monitor. This method MUST update the list of ResourceMonitor instances hold by the Resource Context (getContext().removeMonitor(this)).

*Throws* ResourceMonitorException– For example, when the monitor can't be removed from the Resource-Context.

**144.22.8.2      public abstract void disable() throws ResourceMonitorException**

- Disable the monitoring of this resource type for the resource context associated with this monitor instance. The resource usage is not available until it is enabled again.

*Throws* ResourceMonitorException– if the ResourceMonitor instance has been previously deleted

**144.22.8.3      public abstract void enable() throws ResourceMonitorException**

- Enable the monitoring of this resource type for the resource context associated with this monitor instance. This method SHOULD also update the current resource consumption value (to take into account all previous resource allocations and releases occurred during the time the monitor was disabled).

*Throws* ResourceMonitorException– if the ResourceMonitor instance can not be enabled (for example, some MemoryMonitor implementations evaluate the memory consumption by tracking memory allocation operation at runtime. This kind of Monitor can not get instantaneous memory value. Such Monitor instances need to be enabled at starting time.), if the ResourceMonitor instance has been previously deleted

**144.22.8.4      public abstract boolean equals(Object resourceMonitor)**

*resourceMonitor*

- Checks if resourceMonitor is equals to the current instance. A ResourceMonitor rm1 is equals to a ResourceMonitor rm2 if rm1.getContext().equals(rm2.getContext()) and r1.getType().equals(rm2.getType()).

*Returns* true if the current instance is equals to the provided resourceMonitor

**144.22.8.5      public abstract ResourceContext getContext()**

- Returns the resource context that this monitor belongs to

*Returns* The associated {@link ResourceContext}

**144.22.8.6      public abstract long getMonitoredPeriod()**

- Returns the time period for which the usage of this resource type is monitored.

*Returns* The monitored period in milliseconds, or -1 if a monitored period is not relevant for this resource type.

**144.22.8.7      public abstract String getResourceType()**

- The name of the resource type that this monitor represents

*Returns* The name of the monitored resource type

**144.22.8.8      public abstract long getSamplingPeriod()**

- Returns the sampling period for this resource type.

*Returns* The sampling period in milliseconds, or -1 if a sampling period is not relevant for this resource type.

**144.22.8.9      public abstract Comparable<T> getUsage() throws ResourceMonitorException**

- Returns an object representing the current usage of this resource type by this resource context.

*Returns* The current usage of this resource type.

*Throws* ResourceMonitorException – if the ResourceMonitor instance is not enabled.

#### 144.22.8.10 **public abstract int hashCode()**

- Retrieves the hashCode value of this ResourceMonitor. The hashCode value is based on the hashCode value of the associated ResourceContext and the hashCode value of the type.

*Returns* hashCode

#### 144.22.8.11 **public abstract boolean isDeleted()**

- Returns true if the ResourceMonitor instance has been deleted, that is the {@link #delete()} method has been called previously.

*Returns* true if deleted.

#### 144.22.8.12 **public abstract boolean isEnabled()**

- Checks if the monitoring for this resource type is enabled for this resource context

*Returns* true if monitoring for this resource type is enabled for this context, false otherwise

### 144.22.9 **public class ResourceMonitorException extends Exception**

Resource Monitor Exception reports an invalid usage of a monitor.

#### 144.22.9.1 **public ResourceMonitorException(String msg)**

*msg* message

- Create a new ResourceMonitorException

#### 144.22.9.2 **public ResourceMonitorException(String msg, Throwable t)**

*msg* message

*t*

- Create a new ResourceMonitorException

### 144.22.10 **public abstract ResourceMonitorFactory<T>**

*<T>* The type for the Resource.

A Resource Monitor Factory is a service that provides Resource Monitor instances of a specific resource type (for example, CPUMonitor, MemoryMonitor...) for every Resource Context. Every Resource Monitor Factory service is registered with the {@link #RESOURCE\_TYPE\_PROPERTY} mandatory property. This property indicates which type of Resource Monitor a Resource Monitor Factory is able to create. The type can also be retrieved through a call to {@link #getType()}. The type MUST be unique (two Resource Monitor Factory instances MUST not have the same type).

#### 144.22.10.1 **public static final String RESOURCE\_TYPE\_PROPERTY = "org.osgi.resourcemonitoring.ResourceType"**

Resource type property. The value is of type {@link String}. For example, {@link ResourceMonitoringService#RES\_TYPE\_CPU}

#### 144.22.10.2 **public abstract ResourceMonitor<T> createResourceMonitor(ResourceContext resourceContext) throws ResourceMonitorException**

*resourceContext* ResourceContext instance associated with the newly created ResourceMonitor instance

- Creates a new ResourceMonitor instance. This instance is associated with the ResourceContext instance provided as argument ( {@link ResourceContext#addResourceMonitor(ResourceMonitor)} ) is

called by the factory). The newly ResourceMonitor instance is disabled. It can be enabled by calling {@link ResourceMonitor#enable()}.

*Returns* a ResourceMonitor instance

*Throws* ResourceMonitorException – If the factory is unable to create a ResourceMonitor For example, when a ResourceMonitor of this type already exists for this ResourceContext

#### 144.22.10.3 **public abstract String getType()**

- Returns the type of ResourceMonitor instance this factory is able to create.

*Returns* factory type

### 144.22.11 **public abstract ResourceMonitoringService**

It manages the Resource Context instances. It is available through the OSGi service registry. This service holds the existing Resource Context instances. Resource Context instances are created by calling the {@link #createContext(String, ResourceContext)} method.

#### 144.22.11.1 **public static final String FRAMEWORK\_CONTEXT\_NAME = "framework"**

The name of the special, optional resource context, representing the whole OSGi framework.

#### 144.22.11.2 **public static final String RES\_TYPE\_CPU = "resource.type.cpu"**

The name of the CPU resource type, used to monitor and control the CPU time used by a resource context. {@link ResourceMonitoringService} implementations must create {@link CPUMonitor} instances for this resource type.

#### 144.22.11.3 **public static final String RES\_TYPE\_DISK\_STORAGE = "resource.type.disk.storage"**

The name of the disk storage resource type, used to monitor and control the size of the persistent storage used by a resource context. {@link ResourceMonitoringService} implementations must create {@link DiskStorageMonitor} instances for this resource type.

#### 144.22.11.4 **public static final String RES\_TYPE\_MEMORY = "resource.type.memory"**

The name of the memory resource type, used to monitor and control the size of the java heap used by a resource context. {@link ResourceMonitoringService} implementations must create {@link MemoryMonitor} instances for this resource type.

#### 144.22.11.5 **public static final String RES\_TYPE\_SOCKET = "resource.type.socket"**

The name of the socket resource type, used to monitor and control the number of existing sockets used by a resource context. {@link ResourceMonitoringService} implementations must create {@link SocketMonitor} instances for this resource type.

#### 144.22.11.6 **public static final String RES\_TYPE\_THREADS = "resource.type.threads"**

The name of the threads resource type, used to monitor and control the number of threads created by a resource context. {@link ResourceMonitoringService} implementations must create {@link ThreadMonitor} instances for this resource type.

#### 144.22.11.7 **public static final String SYSTEM\_CONTEXT\_NAME = "system"**

The name of the Resource Context associated with System bundle (bundle 0).

#### 144.22.11.8 **public abstract ResourceContext createContext(String name, ResourceContext template)**

*name* The name identifying the context. Names must be unique within the framework instance.

*template* If a template is provided, the new resource context will inherit all resource monitoring settings (enabled monitors, thresholds) from the template.

- Creates a new {@link ResourceContext}.

A {@link ResourceContextEvent} with type {@link ResourceContextEvent#RESOURCE\_CONTEXT\_CREATED} will be sent.

*Returns* A new {@link ResourceContext} instance.

*Throws* `IllegalArgumentException`—if a problem occurred, for example if the name is already used.

#### 144.22.11.9 **public abstract ResourceContext getContext(String name)**

*name* The resource context name

- Returns the context with the specified resource context name.

*Returns* An existing {@link ResourceContext} with the specified name, or null if such a context doesn't exist

#### 144.22.11.10 **public abstract ResourceContext getContext(long bundleId)**

*bundleId* bundle identifier

- Returns the {@link ResourceContext} associated to the provided bundle id.

*Returns* the {@link ResourceContext} associated to bundle b or null if the bundle b does not belong to a Resource Context.

#### 144.22.11.11 **public abstract String[][] getSupportedTypes()**

- Returns a list with the supported resource type names.

*Returns* An array containing the names of all resource types that this {@link ResourceMonitoringService} implementation supports.

#### 144.22.11.12 **public abstract ResourceContext[][] listContext()**

- Lists all available {@link ResourceContext} resource contexts. The list will contain the special {@link #FRAMEWORK\_CONTEXT\_NAME} context and the {@link #SYSTEM\_CONTEXT\_NAME} context, if it is supported.

*Returns* An array of {@link ResourceContext} objects, or an empty array, if no contexts have been created.

## 144.23 **org.osgi.service.resourcemonitoring.monitor**

Resource Monitoring Monitor Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.resourcemonitoring.monitor; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.resourcemonitoring.monitor; version="[1.0,1.1)"}
```

### 144.23.1 **Summary**

- `CPUMonitor` - A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_CPU} resource type.
- `DiskStorageMonitor` - A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_DISK\_STORAGE} resource type.

- `MemoryMonitor` - A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_MEMORY} resource type.
- `SocketMonitor` - A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_SOCKET} resource type.
- `ThreadMonitor` - A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_THREADS} resource type.

### 144.23.2 **public abstract CPUMonitor** **extends ResourceMonitor<Long>**

A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_CPU} resource type. `CPUMonitor` instance monitors the CPU consumed by a {@link ResourceContext} instance.

#### 144.23.2.1 **public abstract long getCPUUsage()**

- Returns the CPU usage as a cumulative number of nanoseconds

The {@link #getUsage()} method returns the same value, wrapped in a long.

*Returns* the CPU usage in nanoseconds

### 144.23.3 **public abstract DiskStorageMonitor** **extends ResourceMonitor<Long>**

A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_DISK\_STORAGE} resource type. A `DiskStorageMonitor` instance monitors and limits the persistent storage of the bundle belonging to the {@link ResourceContext}

#### 144.23.3.1 **public abstract long getUsedDiskStorage()**

- Returns the sum of the size of the persistent storage areas of the bundles in this resource context.

The {@link #getUsage()} method returns the same value, wrapped in a long.

*Returns* the sum of the sizes of the persistent storage areas in bytes

### 144.23.4 **public abstract MemoryMonitor** **extends ResourceMonitor<Long>**

A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_MEMORY} resource type. A `MemoryMonitor` instance monitors and limits the memory used by a {@link ResourceContext} instance.

#### 144.23.4.1 **public abstract long getMemoryUsage()**

- Returns the size of the java heap used by the bundles in this resource context.

The {@link #getUsage()} method returns the same value, wrapped in a long.

*Returns* the size of the used java heap in bytes

### 144.23.5 **public abstract SocketMonitor** **extends ResourceMonitor<Long>**

A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_SOCKET} resource type. {@link SocketMonitor} instance are used to monitor and limit the number of in-use sockets per {@link ResourceContext} instance. {@link SocketMonitor} instance handle all types of sockets (TCP, UDP, ...).

A TCP socket is considered to be in-use when it is bound ( {@link Socket#bind(java.net.SocketAddress)}) or when it is connected ( {@link Socket#connect(java.net.SocketAddress)}). It leaves the in-use state when the socket is closed ( {@link Socket#close()}). \*

A UDP socket is in-use when it is bound ( {@link DatagramSocket#bind(java.net.SocketAddress)}) or connected ( {@link DatagramSocket#connect(java.net.SocketAddress)}). A UDP Socket leaves the in-use state when it is closed ( {@link DatagramSocket#close()}).

#### 144.23.5.1 **public abstract long getSocketUsage()**

- Returns the number of existing socket created by a {@link ResourceContext}.  
The {@link #getUsage()} method returns the same value, wrapped in a long.

*Returns* the number of existing socket.

#### 144.23.6 **public abstract ThreadMonitor extends ResourceMonitor<Integer>**

A {@link ResourceMonitor} for the {@link ResourceMonitoringService#RES\_TYPE\_THREADS} resource type. A ThreadMonitor instance monitors and limits the thread created by a {@link ResourceContext} instance.

##### 144.23.6.1 **public abstract int getAliveThreads()**

- Returns the number of alive threads created by the bundles in this resource context. A Thread is considered to be alive when its java state is one of the following:
  - RUNNABLE
  - BLOCKED
  - WAITING
  - TIMED\_WAITING

The {@link #getUsage()} method returns the same value, wrapped in a int.

*Returns* the number of alive threads created by this resource context

## 144.24 References

- [1] *Adaptive Monitoring of End-user OSGi based Home Boxes*  
Y. Maurel, A. Bottaro, R. Kopetz, and K. Attouchi. Component Base Software Engineering, 15th ACM SIGSOFT International Symposium on Component-Based Software Engineering, CBSE'2012, Bertinoro, Italy, June 2012.
- [2] *Memory Monitoring in a Multi-tenant OSGi Execution Environment*  
K. Attouchi, G. Thomas, A. Bottaro, and G. Muller. Proceedings of the 17th ACM SIGSOFT symposium on Component Based Software Engineering, CBSE'14, Lille, France, July 2014.

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

**End Of Document**

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

**DRAFT**